

Tarefas de Aprendizagem - Ciclo **while**

As tarefas aqui propostas são para reforçar a sua aprendizagem.

1. Faça uma síntese textual e mapa conceitual da 4ª aula semanal. Defina o que são estruturas de repetição, os tipos de estruturas existentes e o que permitem fazer.
2. Soma de números de 1 a 100: Usando um loop while, escreva um programa que calcula a soma dos números de 1 a 100.
3. Contar de 10 até 0: Usando while, escreva um programa que conta de 10 até 0, utilizando range() para contagem regressiva.
4. Pular múltiplos de 3: Usando while e continue, escreva um programa que exibe números de 1 a 20, mas pula os múltiplos de 3.
5. Encontre o primeiro múltiplo de 7: Usando while, escreva um programa que encontra o primeiro número múltiplo de 7 entre 1 e 50 e interrompe o loop com break.
6. Parar o loop quando o número é 8: Usando while e break, escreva um programa que exibe números de 1 a 10, mas para o loop quando encontrar o número 8.
7. Contagem de dígitos em um número: Usando while, escreva um programa que recebe um número inteiro e conta quantos dígitos ele tem.
8. Somar números até o primeiro negativo: Usando while, escreva um programa que soma números inteiros positivos fornecidos pelo usuário até que um número negativo seja inserido.
9. Pretende-se escrever um pequeno programa que pergunte ao utilizador se “Já chegámos?”. O programa deve repetir a pergunta enquanto a resposta não for “Sim”. DICA: tente tornar o programa menos sensível a letras maiúsculas e minúsculas. O método `str.upper()`, em que `str` é uma *string*, devolve uma *string* com caracteres convertidos para maiúsculas.
10. Crie um programa em Python que apresenta, no ecrã, a lista dos primeiros N quadrados perfeitos (1, 4, 9, 16, 25, ...), onde N é um número natural introduzido pelo utilizador.
11. Altere o programa anterior para que este apresente os primeiros N quadrados perfeitos que **não são múltiplos** de 4.

12. Desenvolva um programa que peça ao utilizador 2 números inteiros e calcule a soma de todos os números inteiros compreendidos entre esses números (com os próprios limites incluídos). O utilizador deve colocar primeiro o menor número e depois o maior número. Por exemplo, se o utilizador introduzir 10 e 14 o resultado deve ser $10+11+12+13+14$.
13. Altere o programa anterior para que este funcione mesmo que o utilizador introduza primeiro o maior número.

Desafios:

14. Crie um programa que imprima no ecrã o seguinte padrão:

```
1
1 2
1 2 3
1 2 3 4
1 2 3 4 5
```

15. Apresente um programa que mostre os primeiros N termos da sequência de Fibonacci. Nessa sequência, cada elemento é igual à soma dos dois elementos imediatamente anteriores. Considere que a sequência começa com 0 e 1, ou seja:

0 1 1 2 3 5 8 13 21 ...

16. Crie um programa que imprima no ecrã o seguinte padrão:

```
1 2 3 4 5
1 2 3 4
1 2 3
1 2
1
```

17. Crie um programa para verificar se o número que um utilizador introduz é primo (um número primo é um número natural maior do que 1 que só é divisível por 1 e por ele próprio).

Bons estudos!